

Knowledge Representation

by Adia Khalid

Summer semester 2026

Lecture notes by Liliana Sanfilippo

April 2026

Contents

I	Foundation of Knowledge Representation	7
1	Introduction to KR: Definition, Scope, Motivation (Use Cases), and History	9
1.1	Introduction	9
1.2	Taxonomy of Representation Schemes	11
1.3	Distinguishing Features of KR Systems	14
1.4	Applications of Knowledge Representation	17
1.5	Future Directions	19
1.6	Conclusion <i>brought to you by claude.ai</i>	23
1.7	Course structure	23
1.8	Formalities	23
2	Logical Foundations: First-Order Logic (FOL)	25
2.1	Syntax	25
2.2	Semantics	26
2.3	Inference	27
2.4	Proof Systems & Completeness	27
3	Logic Programming and Automated Reasoning: Prolog as a Knowledge Representation Language	29
3.1	Why Prolog for KR?	29
3.2	Prolog and First-Order Logic	30
3.3	Prolog Syntax	30
3.4	A Running Example: Family Relations	31
3.5	How Prolog Executes: Matching and Search	31
3.6	Declarative vs. Procedural Meaning	31
3.7	Lists	31
3.8	Recursion and Arithmetic	32
3.9	Clause Ordering and Loops	32
3.10	The Monkey and Banana Problem	32
3.11	Backtracking, Cuts, and Negation	32
3.12	Horn Clauses and Proof	32
3.13	Operators	32
4	Structured Knowledge Representation	33
4.1	Types of Knowledge	33
4.2	Frames and Slots	34
4.3	Object-Oriented Analogy: Classes and Instances	34
4.4	Inheritance Hierarchies	34
4.5	Scripts and Semantic Networks	34

4.6	Default Reasoning and Conditionals	34
4.7	Problems with Schema Systems	34
4.8	Frames in Practice: JSON-LD	34
5	Semantic Network Analysis (SemNA)	35
5.1	Motivation	35
5.2	Key Definitions	36
5.3	Problem Statement	38
5.4	Proposed Solution: the SemNA pipeline	38
5.5	Network Measures	38
5.6	Results	38
5.7	Limitations & Discussion	38
5.8	Discussion & Conclusion	38
6	The Resource Description Framework	39
6.1	What is RDF?	39
6.2	Triple Model	39
6.3	International Resource Identifiers (IRIs/URIs)	40
6.4	Literals	40
6.5	Standard Format Turtle - Terse RDF Triple Language	41
6.6	Standard Format JSON	41
6.7	Flat Triplet Approach to RDF Graphs in JSON	41
6.8	Comparison of Standard RDF Formats	41
6.9	Summary	41
7	Knowledge Graph Reasoning	43
7.1	Motivation and Background	43
7.2	Background Knowledge	44
7.3	Inductive Logic Programming-Based KGR Methods	44
7.4	Probabilistic Graphical Models + Rules	44
7.5	Embedding Techniques + Rules	45
7.6	Neural Networks + Rules	45
7.7	Comparison and Analysis	45
7.8	Challenges and Future Directions	46
8	Knowledge Aquisition from Data I	49
9	Knowledge Aquisition from Data II	51
II	Knowledge Graphs	53
10	Start into semantic networks and knowledge graphs reasoning	55
11	The Resource Description Framework (RDF)	57
12	Knowledge Graph Reasoning	59
13	Knowledge Acquisition from Data	61
14	Representation Learning	63

III Knowledge Representation in Machine Learning	65
15 Knowledge Graph Embeddings	67
16 Graph Neural Networks for Knowledge Representation	69
IV Applications	71
17 Knowledge Tracing	73
18 Knowledge in Large Language Models	75
19 KG- Aware Applications	77
Backmatter	i
Index	i
Definitions	i
References	vii

Part I

Foundation of Knowledge Representation

CHAPTER 1

Introduction to KR: Definition, Scope, Motivation (Use Cases), and History

1.1 | Introduction

What is Knowledge Representation?

Definition 1.1 Knowledge Representation

KR is a central problem in Artificial Intelligence that focuses on developing sufficiently precise notations for representing knowledge.

The core problem with KR is that the real world with its complexity, ambiguity and dynamism can never be fully represented in a computer-based system. Nevertheless, intelligent systems must be able to function effectively using partial, incomplete and sometimes contradictory information. [9]

Why is Knowledge Representation needed?

The problem of data meaninglessness

To understand the need for KR, let's consider a practical scenario:

Example 1.1 A student database.

A hypothetical database contains the following entries:

Name	Major	CGPA	Research	...
Alice Smith	CS	3.8	Genomics	...
Bob John	Biology	3.8	Oncology	...
Carol Li	Zoology	3.7	Animal colonisation	...

Without knowledge representation, this data is disjointed. The entries exist in isolation across thousands of files. A database query system can only perform exact keyword matches:

- Query 1: "Show me all AI researchers" → No results (not explicitly mentioned in the data)
- Query 2: "Which students could work on this patent?" → No answer possible
- Query 3: "What are the emerging trends?" → No analysis possible

Data is meaningless without context and relationships. It does not allow for intelligent reasoning or deeper analysis.

What knowledge representation makes possible

With knowledge representation, we can structure the data and transfer isolated records into interconnected, queryable knowledge using:

- **ontologies:** Explicitly specify which concepts exist (student, researcher, field of research) and how they are categorised.
- **Knowledge graphs:** Link entities together: Alice $\rightarrow$ researcher $\rightarrow$ AI

This allows for semantic searches, automatic relationship discovery and reasoning across all connection.

Example 1.2 ■ Semantic search

- “Find students researching machine learning?”
- “Show me all AI researchers”: The system can use transitive relationships to find relevant students.
- “Which students are a good fit for this patent?": The system can match skills and areas of interest.
- **Automatic relationship discovery**
 - “What are emerging research trends?”
 - Recognising patterns across all data
 - Identifying emerging trends through interconnected information

Data

Raw, unprocessed facts

e.g. CGPA: 3.8

Information

Structured and contextualised data

e.g. Alice has a CGPA of 3.8 in Computer Science

Knowledge

Information combined with experience and reasoning rules
e.g. Alice could work on this AI patent because she has a strong foundation in computer science and is interested in genomics

Table 1.1: Data vs. Information vs. Knowledge

KR enables machines to make informed decisions with fewer rules.

KR Perspective

Traditional ML systems follow a black-box approach which means the decision-making process is not transparent and the reasoning behind predictions is not accessible.

- **Generalisation:** Requires continuous retraining with new data
- **Explainability:** Difficult or impossible to explain decisions
- **Data flexibility:** Highly dependent on large amounts of training data

Knowledge-Based AI utilises structured, **explicit knowledge**.

Definition 1.2 explicit knowledge

Knowledge that is formally encoded in symbols, rules or graphs and can be processed by machines.

- **Generalization:** explicit knowledge transfers to new problems without retraining from scratch[6]
- **Explainability:** encoded knowledge enables transparent reasoning and human-understandable decisions[2]
- **Data flexibility:** Modifiable

Representation Schemes

Definition 1.3 representation scheme

A representation scheme is a formal notation used to define a knowledge base (Mylopoulos 1980). A knowledge base contains facts and rules and serves as a model of the real world. A representation scheme is therefore the ‘language’ in which we express knowledge, much like programming languages define the syntax and semantics in which we write instructions..

Example 1.3 Pac-Man ghost AI has a knowledge-based approach: We code ghost behaviour as explicit rules.

1. IF player_nearby THEN chase
2. IF frightened THEN flee
3. ELSE patrol

Decisions are transparent, explainable, and modifiable without recomputing. (Ogland 2016)

Challenges in Knowledge Representation

Knowledge representation involves designing formal structures that allow computers to store, interpret, and reason about information and every design decision has advantages and caveats.

Choosing the representation is one of the design challenges. **Semantic networks** are intuitive and visually understandable but have limited inference. **First-order logic** on the other hand is expressive, but has computationally costly reasoning.

Desired properties can conflict. Systems should ideally be expressive enough to capture rich real-world knowledge while remaining efficient enough to reason in practical time. Tradeoffs are between expressiveness and tractability, and Human readability and machine efficiency.

Application-driven constraints arise from, different domains requiring different forms of representation. E.g. In natural language processing (NLP), systems must handle ambiguity and context. In medical diagnosis, representations must manage uncertainty, causal relationships, and strict correctness requirements. In question answering, systems need fast retrieval combined with reasoning over large amounts of structured and unstructured data

The core idea of Knowledge Representation is not just a data structure. It plays multiple roles simultaneously and those roles can conflict with each other:

- Surrogate for the world: Acts as a substitute for real world objects and relationships
- Ontological commitments: Defines what exists and how things are categorized
- Theory of reasoning: Provides foundation for logical inference and deduction
- Medium for efficient computation: Enables performant computational processing
- Medium of Human expression: Allows humans to express knowledge in understandable ways

1.2 | Taxonomy of Representation Schemes

Representational schemata can be divided into four basic categories:

- Declarative representations
 - Logical representation schemes
 - Semantic (network) schemes
- Procedural representations
- Hybrid approaches
- Frame-based representations

Declarative representations

Definition 1.4 declarative representation

Declarative representations describe facts about the world as logical statements.

Declarative representations focus on truth and semantics and enable deductive reasoning. The schema is independent of how the knowledge is used.

There are two subtypes:

1. Logical representation schemes

Definition 1.5 logical representation scheme

Representation schema based on formal logic and represents knowledge through logical formulas. It is strongly based on semantics and inference.

Example 1.4 First order Logic



Figure 1.1: This is Tweety. What can we infer about Tweety with the information given below? (image from <https://www.pexels.com/photo/pink-cockatoo-perched-in-natural-habitat-29732587/>)

We know that all bird can fly if they are not flightless birds, here generalized as penguins. We know that Tweety is a bird and not a penguin.

More formal:

Birds can fly: $\forall x(Bird(x) \wedge \neg Penguin(x) \rightarrow CanFly(x))$

Facts: $Bird(Tweety)$ and $\neg Penguin(Tweety)$

Therefore we can infer:

$CanFly(Tweety)$

2. Semantic (network) schemes

Definition 1.6 semantic representation scheme

Graph-based representations in which nodes represent entities and edges represent relationships between them. It supports inheritance and classification.

Example 1.5

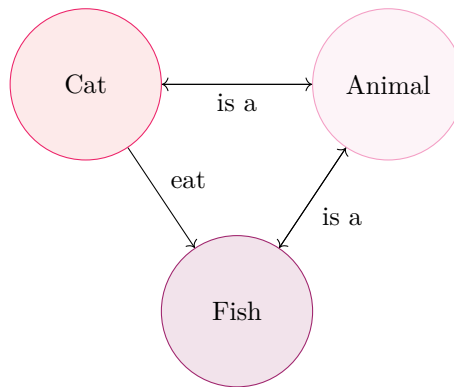


Figure 1.2: Semantic Network

Procedural representations

Definition 1.7 procedural representation

Representation schema that encodes knowledge as procedures or programmes that describe how to use that knowledge. It is efficient but hard to interpret.

Example 1.6



Figure 1.3: This is Dr. Dokter. She wants to diagnose her patient. (image from <https://www.pexels.com/photo/female-doctor-in-scrub-suit-7659874/>)

The rules are:

IF (patient has fever) **AND** (patient has cough) **AND** (patient has difficulty breathing) **THEN diagnose:**
possible pneumonia, **recommend** chest x-ray

This allows Dr. Dokter to properly diagnose her patients instead of furthering the antibiotic resistant strand of

bacteria!

Hybrid approaches combine declarative and procedural aspects.

Frame-Based Representation

Definition 1.8 frame

Concept designed to provide a structured way to capture the essential aspects of a situation, facilitating easier retrieval and manipulation of information. They are akin to schemas or blueprints that organize knowledge into manageable chunks. Attributes of the represented concept are represented by slots which have facets and default values.

Example 1.7 Back to Tweety from fig. 1.1.

```

Frame: Bird
  is_a:      Animal
  has_parts: [wings, beak, feathers]
  can:       fly
  default_color: pink
  locomotion: flying

Frame: Penguin
  is_a:      Bird
  can:       swim
  cannot:    fly (overrides parent default)
  
```

1.3 | Distinguishing Features of KR Systems

Unlike databases (retrieval) or programs (execution), knowledge bases reuse the same facts in multiple ways:

Inference (rule-based reasoning):

Facts are used to derive new facts by applying rules.

Example 1.8 We remember Dr. Dokter from fig. 1.3.

IF fever $\wedge$ cough $\rightarrow$ possible infection

$\Rightarrow$ Diagnostic conclusion

It has multiple subtypes:

Definition 1.9 logical inference

Inference that uses formal deduction from known facts.

Example 1.9

IF fever $\rightarrow$ infection
fever $\Rightarrow$ infection

Definition 1.10 specialized procedures

Fixed inference patterns, which are often implemented as routines.

Example 1.10 Ward A $\rightarrow$ Ward B $\rightarrow$ ICU $\Rightarrow$ Shortest path

Definition 1.11 preferred inference

Reasoning based on typical assumptions, subject to certain reservations.

Example 1.11

Birds fly
Tweety is a bird $\Rightarrow$ Tweety flies
(except penguins)

Definition 1.12 inductive inference

Generalising examples into patterns or rules.

Example 1.12

Data $\Rightarrow$ Pattern
Symptoms $\Rightarrow$ Diagnosis model

Definition 1.13 abductive inference

Reasoning to arrive at the best explanation. From observations to hypothetical causes.

Example 1.13

Observed: fever + cough
 $\Rightarrow$ Best explanation: pneumonia

Access (Direct fact retrieval via queries):

Facts are retrieved directly, much like in a database.

Example 1.14 Query: “Which patients have fever?” $\Rightarrow$ Retrieve patient X

Matching (Similarity-based reasoning):

Facts are used to identify similar patterns and perform case-based reasoning.

Example 1.15

$X \sim \text{Patient Y} \Rightarrow \text{Case-based reasoning}$

Design implications

When designing a system, we must choose which inference mechanisms to support (e.g., shortest-path search vs. rule-based reasoning). Of course, there are trade-offs between expressiveness and efficiency. And different applications

require different inference types, e.g. the Pac-Man game (example 1.3) needs heuristic inference for real-time decisions goals.

Observation 1.1 Knowledge bases cannot completely describe the world they model (except artificial microworlds)

Consequences exist for operations, reasoning and design methodology.

Consequences for Operations

Implication 1.1 Regarding observation 1.1

There are different answers to existence queries depending on assumptions.

We can assume a closed world, where everything not explicitly stated in the knowledge base is assumed FALSE or an open world, where we simply cannot conclude anything if it is not explicitly stated in the knowledge base.

Example 1.16 To definition ?? - Closed World Assumption

Knowledge Base: Alice (New York), Bob (London), Carol (Paris)

Query: "Is there a person in Toronto?"

CWA Answer: NO (because Toronto is not mentioned in the KB)

Reasoning: If someone lived in Toronto, the KB would state it.

Example 1.17 To definition ?? - Open World Assumption

Knowledge Base: Alice (New York), Bob (London), Carol (Paris)

Query: "Is there a person in Toronto?"

OWA Answer: Unknown (because Toronto is not mentioned, but also not excluded)

Reasoning: The KB does not mention Toronto, but that does not mean no one lives there.

The choice between CWA and OWA fundamentally influences system behavior and must be made during the design phase.

Consequences for Reasoning

Incompleteness requires special reasoning strategies:

- Assumption of Defaults: In the absence of information, make reasonable assumptions
- Probabilistic Reasoning: Explicitly model uncertainties
- Constraint Propagation: Use constraints to reduce search spaces

Consequences for Design Methodology

Unlike machine learning, where knowledge is implicit in model parameters and retraining is required, in KGs knowledge is explicit and can be extended, revised, and corrected without retraining the entire model.

This distinction is important for practical systems where expert knowledge needs to be continuously updated without expensive computational overhead.

Programming	Knowledge Representation	Machine Learning
“Once and for all” complete specification. Code is written and then “frozen”	Continuous, incremental development. The knowledge base is iteratively extended, revised, and refined	Knowledge is implicitly encoded in model parameters and requires retraining when incomplete

Table 1.2: Differences in knowledge/information acquisition and handling.

Definition 1.14 Knowledge Acquisition Processes

The process of incorporating knowledge into the knowledge base through strategies such as Interactive sessions with experts or Automatic generation from experience.

1.4 | Applications of Knowledge Representation

“The KR discipline has developed in waves, with each era developing different approaches to different problems.”

– *claude.ai*

Definition 1.15 test
todo.

Example 1.18 To definition 1.15 - MYCIN’s Approach

Domain: Infectious blood diseases and bacterial infections

Knowledge Base Structure:

- Approximately 600 production rules
- Rules encoded expert medical knowledge
- Format: IF [conditions] THEN [conclusions/actions]

Definition 1.16 Natural Language Processing
todo.

Natural Language Processing includes Semantic understanding, Machine translation, Question answering and Dialogue systems.

Example 1.19 For Natural Language Processing:

- Semantic understanding - “Paris is the capital of France” $\Rightarrow$ understands (`capital_of(Paris, France)`)
- Machine translation - “The doctor treated the patient” $\Rightarrow$ preserves roles across languages (doctor = agent, patient = object)
- Question answering - Q: “Who is the capital of France?” $\Rightarrow$ Query knowledge graph Answer: Paris
- Dialogue systems - User: “I have a fever.” System: “Do you also have a cough?” $\Rightarrow$ Uses context and stored knowledge

Today (in the modern era), we use **knowledge graphs**.

Definition 1.17 knowledge graph

Graph-based knowledge representation where the entities are nodes and the relations are edges. Facts (subject, a predicate, and an object) are represented as triples..

Example 1.20 To ??

- Google Knowledge Graph (search enhancement) Example: Search “Albert Einstein” → panel shows: born in Ulm, Nobel Prize, physicist, spouse Mileva Maric
- Knowledge Vault (knowledge fusion) Example: Automatically merges facts from Wikipedia, Freebase, and news articles into unified knowledge base
- Enterprise Knowledge Management Example: Company stores employee skills, projects, teams → helps find experts for new projects
- Recommendation Systems Example: Amazon recommends “Laptop Bag” because you bought “Laptop” → relationship stored in knowledge graph

Knowledge graphs are used for better information retrieval. E.g. for **Information Retrieval (Semantic Search)**s.

Definition 1.18 Information Retrieval (Semantic Search)

todo

Example 1.21 Query: “treatment for viral pneumonia”

Without KG: Documents containing “treatment” + “viral” + “pneumonia”

With KG:

pneumonia –[caused_by]–> virus virus –[type]–> COVID-19, Influenza, etc. COVID-19 –[treated_by]–> antiviral drugs, ventilation

Retrieval: Documents about COVID-19 treatment, antiviral therapies (not necessarily containing the word “pneumonia”)

Knowledge representation techniques are also used in database design for semantic data models, query optimization, constraint management and data integration.

Example 1.22 Semantic Data Models: using an ER diagram to represent Students, Courses, and Enrollments. *Shoutout to claude*

Entities:

- Student (ID, Name, Major, CGPA)
- Course (Course_ID, Name, Credits)
- Enrollment (Student_ID, Course_ID, Grade)

Relationships:

- Student enrolls_in Course
- Course belongs_to Department

Example 1.23 Query Optimization - creating an index on `student_id` to speed up `SELECT` queries.

Shoutout to claude

Query: `SELECT student_name FROM Student WHERE enrolled_in_course(CS101)`

Plan A (inefficient):

1. Load all students from disk
2. For each student, check all enrollments
3. Filter for CS101

Plan B (efficient):

1. Use index on (`Course_ID`)
2. Find all enrollments where `Course_ID = CS101`
3. Load corresponding student records

Example 1.24 Constraint Management - enforcing primary key and foreign key constraints to maintain data integrity. *Shoutout to claude*

Primary Key Constraint: Each student has a unique ID

Foreign Key Constraint: Each enrollment must reference a valid `student_ID`

Domain Constraint: CGPA must be between 0.0 and 4.0

Business Rule: A student cannot take more than 5 courses per semester

Further, **Knowledge Representation** can be used for program specifications and system interfaces.

Example 1.25 Space Tutor, Folie 37

1.5 | Future Directions

Multi-hop Reasoning (Complex Reasoning)

Many questions require multiple reasoning steps across relationships.

Example 1.26 Query: “What medicine treats diseases caused by virus X?”

Requires:

medicine $\rightarrow$ [treats] $\rightarrow$ disease $\rightarrow$ [caused_by] $\rightarrow$ virus

Path:

Starting from virus_X:

virus_X $\rightarrow$ [causes] $\rightarrow$ disease_Y

disease_Y $\rightarrow$ [treated_by] $\rightarrow$ medicine_Z

Result: medicine_Z

Simple embedding methods struggle with this multi-step logical chain, so complex multi-hop relations require dedicated design.

Promising Directions:

- **Recurrent Relational Path Encoding:**

- Methods like DeepPath use LSTM networks
- The LSTM “encodes” a path like: $\text{Person} \rightarrow \text{LivesIn} \rightarrow \text{City} \rightarrow \text{LocatedIn} \rightarrow \text{Country}$
- This encoded path representation can then be used for reasoning

■ GNN-based Message Passing:

- Graph Neural Networks (GNNs) aggregate information from neighbors
- Multi-hop reasoning through iterative message passing
- Each “hop” collects information from the next node

■ Reinforcement Learning-based Path Finding:

- An agent learns to navigate through the knowledge graph
- Example agent path: “Marie Curie $\rightarrow$ BornIn $\rightarrow$ Ulm $\rightarrow$... $\rightarrow$ Paris”
- The agent receives rewards when reaching the correct target

Definition 1.19 one-hop

todo.

Definition 1.20 multi-hop

todo.

Example 1.27 To definitions 1.19 and 1.20 - Simple vs. Multi-hop Reasoning

A simple 1-hop query can be expressed as:

$$\text{Einstein} \rightarrow \text{worked_at} \rightarrow \text{Princeton}$$

A more complex multi-hop query is: *Where did Einstein’s doctoral advisor teach?*

This requires chaining relations:

$$\text{Einstein} \rightarrow \text{advised_by} \rightarrow \text{Kleiner} \rightarrow \text{worked_at} \rightarrow \text{University of Zurich}$$

This corresponds to a 2-hop reasoning process.

Structured knowledge provides commonsense observations and acts as relational inductive biases (assumptions about the world). Meaning they provide such biases like: “Some relations are more likely than others”. It also boosts commonsense knowledge fusion between symbolic and semantic space. E.g.:

Example 1.28 Query: *Can a fish play piano?*

Without Knowledge: A machine learning model could predict this based on training data, but provide no reasoning.

With Knowledge:

$$\begin{aligned} \text{Fish} &\rightarrow \text{hasBodyPart} \rightarrow \text{Fins} \quad (\text{not Hands}) \\ \text{PlayingPiano} &\rightarrow \text{requires} \rightarrow \text{Hands} \end{aligned}$$

Symbolic Rule: If $X \rightarrow \text{hasBodyPart} \rightarrow p$ and $\text{action} \rightarrow \text{requires} \rightarrow p$, then $X \rightarrow \text{can} \rightarrow \text{action}$.

Application:

$\text{Fish} \rightarrow \text{hasBodyPart} \rightarrow \text{Fins}$
 $\text{PlayingPiano} \rightarrow \text{requires} \rightarrow \text{Hands}$
 $\text{Fins} \neq \text{Hands}$

Conclusion:

$\text{Fish} \rightarrow \text{cannot} \rightarrow \text{PlayPiano}$

Answer with Reasoning: No, a fish cannot play piano because it lacks hands.

Commonsense knowledge provides strong priors, enabling systems to function with less training data.

Unified Framework

Many KR embedding methods are mathematically equivalent under transformation. Actually, several representation learning models are verified as equivalent.

Example 1.29 Example: Unifying Knowledge Graph Embedding Models

HolE: $h \star r$ (circular correlation)
 ComplEx: complex-valued embeddings
 DistMult: real-valued symmetric embeddings

These can be understood as special cases of a more general framework.

ANALOGY Framework:

The ANALOGY framework unifies these approaches via analogical inference:

$$h + r \approx t$$

where an entity h combined with a relation r approximates the target entity t .

Interpretation: Different models correspond to different ways of representing this analogy.

Interpretability

Modern neural-KR systems (KG embedding models) work well, but it's unclear *why*.

Example 1.30 Example: Lack of Interpretability in Neural Models

Prediction: Neural model predicts:

$\text{Aspirin} \rightarrow \text{treats} \rightarrow \text{Headache}$

Question: Why? Which reasoning path?

Reasoning: Just learned embedding similarity.

There are different ways this could have been reasoned:

■ Path-based explanation:

$\text{Aspirin} \rightarrow \text{type} \rightarrow \text{Painkiller} \rightarrow \text{treats} \rightarrow \text{Pain} \rightarrow \text{symptom_of} \rightarrow \text{Headache}$

■ Attention visualization:

Model focuses on “anti-inflammatory” and “pain-relief” relations

■ **Rule extraction:**

- IF drug $\rightarrow$ has_property $\rightarrow$ analgesic AND disease $\rightarrow$ has_symptom $\rightarrow$ pain
- THEN drug $\rightarrow$ treats $\rightarrow$ disease

And different applications:

■ **Medical diagnosis:**

Doctor needs to see reasoning path before trusting AI recommendation

■ **Legal applications:**

Must explain why system reached a conclusion

Scalability

Knowledge graphs can be extremely large which leads to a trade-off between and computational efficiency and model expressiveness.

Limited amount of works were applied to/tested on more than 1 million entities and inference over probabilistic logic is computationally intensive.

Knowledge Aggregation

Knowledge aggregation is the core of knowledge-aware applications. Many approaches rely on neural architectures. NLP has been significantly improved by large-scale pre-training. A key challenge remains the efficient and interpretable aggregation of knowledge.

Example 1.31 Query: Who directed Inception?

Knowledge neighbors:

[Nolan, Warner Bros, 2010, Sci-Fi, DiCaprio]

Attention weights:

director = 0.8, actor = 0.15, studio = 0.05

Interpretation: The model focuses on the most relevant relation (director).

Automatic Construction and Dynamics

Knowledge graphs are currently mostly manually constructed or extracted from structured sources (e.g. Wikipedia), which is very labor-intensive.

We would need automatic construction from large-scale unstructured content, but there are several challenges:

- **Multimodality:** Information can be in text, images, video
- **Heterogeneity:** Different sources use different schemas
- **Large-scale:** Billions of documents

Additionally, **knowledge graphs** should be dynamic, as real-world facts change over time.

Example 1.32 Static representation:

(Pluto, classified_as, Planet)

Dynamic representation:

(Pluto, classified_as, Planet, [1930-2006])

(Pluto, classified_as, Dwarf_Planet, [2006-present])

Requirement: System must track temporal validity of facts.

1.6 | Conclusion brought to you by *claude.ai*

“Knowledge representation remains a central discipline in artificial intelligence. While the field has historically undergone several paradigm shifts (from pure symbolic AI to knowledge graphs to hybrid systems), the fundamental motivation remains unchanged: How can we structure complex knowledge in a form that machines can understand and use?

The answer continues to evolve. Modern systems combine:”

- Expressiveness of logical systems
- Efficiency of algorithmic approaches
- Scalability of big data techniques
- Learning capability of neural networks
- Interpretability through explicit knowledge

1.7 | Course structure

foundation of KR

Knowledge graphs

KR in mL

grading:

50% of the exercise points (full portfolio) (Abgabe in 3er Gruppen(?)) present one exercise in tut present one paper in seminar 40–45 min (3er gruppen) final report 5–10 pages (50% marks on final report)

Es wird ein Moodle geben

1.8 | Formalities

CHAPTER 2

Logical Foundations: First-Order Logic (FOL)

Knowledge representation requires a formal system that expresses knowledge precisely and derives consequences from that knowledge.^[5]

The most relevant formal calculations are **propositional Logic** and **first-order logic**.

Definition 2.1 **propositional Logic**
todo.

Example 2.1 “If a person is rich, they have a nice car”
That statement becomes

$$\begin{aligned} BobIsRich &\Rightarrow BobHasNiceCar \\ JaneIsRich &\Rightarrow JaneHasNiceCar \\ &\vdots \end{aligned}$$

Having to define every rule for every person is impractical and unintuitive though.

2.1 | Syntax

- 6 - Objects sind x,y, ...8 - not a well-formed formula weil es keinen Sinn macht
- 14 - unary predicate, welche gibt es noch??

Definition 2.2 **formula**
todo.

Definition 2.3 **atomic formula**
todo.

Definition 2.4 **compound formula**
todo.

Definition 2.5 **quantified formula**
todo.

Example 2.2 **Building a Formula – Example from OLP**

Definition 2.6 **free**

todo.

Definition 2.7 **bound**

todo.

Example 2.3 Von f 18**Definition 2.8** **sentence**

todo.

Example 2.4 Von f 19**Definition 2.9** **substitution**

todo.

Example 2.5 Von f 20.1**Definition 2.10** **captured**

todo.

Example 2.6 Von f 21

2.2 | Semantics

Definition 2.11 **language structure**

todo.

Definition 2.12 **domain**

todo.

Definition 2.13 **valid**

todo.

Definition 2.14 **satisfiable**

todo.

Definition 2.15 **unsatisfiable**

todo.

2.3 | Inference

Definition 2.16 inference

todo.

Definition 2.17 Modus Ponens

todo.

Definition 2.18 Modus Tollens

todo.

Definition 2.19 Universal Instantiation

todo.

Definition 2.20 Existential Instantiation

todo.

Definition 2.21 Prenex Normal Form

todo.

F 38 in step 1: Rule 1: $A \rightarrow B = \neg A \cup B$

Rule 2: $\neg \forall x(P(x)) \equiv \dots$ Ist es hier neg oder sim ??

Definition 2.22 clause

todo.

Definition 2.23 literal

Basic Data values that are not international resource identifier (IRI)s.

Definition 2.24 Conjunctive Normal Form

todo.

Definition 2.25 Resolution-based prover

todo.

Definition 2.26 resolution

todo.

2.4 | Proof Systems & Completeness

Definition 2.27 Natural Deduction

todo.

Definition 2.28 Sequent Calculus

todo.

Definition 2.29 derivability relation
todo.

CHAPTER 3

Logic Programming and Automated Reasoning: Prolog as a Knowledge Representation Language

Definition 3.1 Prolog

todo.

3.1 | Why Prolog for KR?

Prolog is usually taught as a programming language, but it is also a knowledge representation language.

We can view programs (also prolog) from two perspectives which both apply.

Declarative / KR view is that a program is a set of logical definitions. You describe what is true, not how to compute it.

Procedural view is that a program is a set of instructions that Prolog executes to answer queries step by step.

Since both views apply to the same program, this means the same program is both knowledge representation and executable reasoning.

Comment 3.1 facts represent anything that could be true

When you write a Prolog program as a set of logical definitions, the same rules can be used for many tasks, not just answering a fixed query.

Definition 3.2 deduction

todo.

Definition 3.3 abduction

todo.

Definition 3.4 containment testing

todo.

Definition 3.5 view differentiation

todo.

Definition 3.6 query folding
todo.

None of these are straightforward if you treat the program as pure code.

3.2 | Prolog and First-Order Logic

Basically Prolog works the same as fol, we have predicates and rules, but it is not the same! Prolog was designed as a direct computational realization of first-order predicate logic. Each concept in FOL has an exact counterpart in Prolog:

FOL concept	Prolog equivalent
Predicate symbol	predicate name, e.g. parent, mortal
$\forall x$	variable (any uppercase name)
Conjunction $\wedge$	comma ‘,’ in rule body
Implication $A \rightarrow B$	rule B :- A.
Ground atom (no variables)	fact
Horn clause	any Prolog clause

This is why reading a Prolog program declaratively is reading logic.

Comment 3.2 F8: ":-" ist a headsign(neck)

Comment 3.3 "? - mortal (socrates) ." ist the query

Definition 3.7 horn clause
todo.

Example 3.1 F8

FOL is descriptive but not executable and therefore needs external theorem provers. Prolog provides built-in inference and supports automatic query answering.

3.3 | Prolog Syntax

Before writing programs, we need to know what Prolog data looks like. Everything in Prolog is a term. There are four kinds:

Definition 3.8 atom
todo.

Definition 3.9 compound term
todo.

Example 3.2 clauses F 12

Definition 3.10 number (prolog)
todo.

Definition 3.11 variable (prolog)
todo.

A Prolog program consists of clauses. There are three kinds:

Definition 3.12 **fact (prolog)**
 todo.

Example 3.3 clauses F 11

Definition 3.13 **rule (prolog)**
 todo.

Example 3.4 clauses F 11

Definition 3.14 **query (prolog)**
 todo.

Example 3.5 clauses F 11

Notation 3.1 Comma , in a rule body means AND. The neck :- means IF

Definition 3.15 **identity of a functor**
 todo.

3.4 | A Running Example: Family Relations

To learn Prolog syntax and reasoning, we will use one concrete example throughout the next several slides: a family tree.

The family tree contains these people:

3.5 | How Prolog Executes: Matching and Search

3.6 | Declarative vs. Procedural Meaning

3.7 | Lists

We need to specify head and tail for every list if we want to do smth like `append ([ H | T1 ] , List2 , [ H | T3 ])` :-

Example 3.6 Extra

$$A = [\underbrace{1, 2}_H, \underbrace{3, 4}_T] B = [\underbrace{d}_H, \underbrace{e, f}_T]$$

If we now do that:

$$\text{append}([\underbrace{H_1}_1 | \underbrace{T_1}_3], B, [\underbrace{H_2}_2 | \underbrace{T_2}_4]) :-$$

we get

$$[1, 3, d, e, f, 2, 4]$$

Comment 3.4 built-in = pre defined

3.8 | Recursion and Arithmetic

3.9 | Clause Ordering and Loops

3.10 | The Monkey and Banana Problem

3.11 | Backtracking, Cuts, and Negation

3.12 | Horn Clauses and Proof

3.13 | Operators

CHAPTER 4

Structured Knowledge Representation

4.1 | Types of Knowledge

Different levels of knowledge that are central to understanding how people learn and reason:

Definition 4.1 declarative knowledge

“Knowing **that**”. Factual and symbolic knowledge used for communication and abstract reasoning..

Definition 4.2 structural knowledge

“Knowing **why**”. Knowledge of how concepts within a domain are interrelated. This mediates declarative to procedural knowledge.

Definition 4.3 procedural knowledge

“Knowing **how**”. Compiled and action-oriented knowledge dependent on structural interconnections.

Example 4.1 Examples for 4.1, 4.2 and ??

- Declarative knowledge: warm air rises
- Structural knowledge: heat $\rightarrow$ low density $\rightarrow$ buoyancy
- Procedural knowledge: performing a convection calculation

What is Structural Knowledge?

“ In order to know how, you must know why. Structural knowledge provides the conceptual bases for why; it describes how the declarative knowledge is interconnected ”

(Jonassen, Beissner, and Yacci 1993)

According to Jonassen, Beissner, and Yacci (1993), the meaning of a concept is implicit in its pattern of relationships to other concepts. Therefore, structural knowledge is closely related to conceptual knowledge, internal connectedness, and integrative understanding. Tennyson and Cocchiarella (1986) define conceptual knowledge as the integrated storage of meaningful dimensions within a specific knowledge domain. Structural knowledge also plays an important role in learning because it mediates the acquisition of procedural knowledge by connecting facts and concepts to actions and procedures.

Why Not Just First-Order Logic?

First-order logic quantifies over individual objects only, not over predicates or sets. This creates fundamental expressive limits:

- Cannot quantify over predicates or sets. Anything about sets must be rephrased in terms of predicates, which is often not possible (Quora contributors, 2024).
- Cannot define mathematical induction in a single finite sentence. Induction requires quantifying over predicates — a second-order principle (Wikipedia contributors, 2026b; Logic and Proof contributors, 2024).
- Cannot express “finitely many” or characterise finiteness/countability: by the Löwenheim–Skolem theorem, any consistent FOL theory with an infinite model also has a countable model (Wikipedia contributors, 2026b).
- No natural grouping of properties around a concept, no built-in defaults, and no context (Steels, 1979).

Lemma 4.1 FOL requires an axiom schema FOL cannot write a single axiom $\forall P.(P(0) \wedge \forall k.(P(k) \rightarrow P(k+1))) \rightarrow \forall n.P(n)$ because $\forall P$ quantifies over predicates. Instead, FOL needs one axiom per predicate (wiki 2026b).

Theorem 4.1 Constructing a knowledge representation

“What we do in constructing a knowledge representation language is to make an abstraction over a class of representational structures and introduce a syntactic mechanism to represent that abstraction.”
(Steels 1979)

4.2 | Frames and Slots

Definition 4.4 frame

Concept designed to provide a structured way to capture the essential aspects of a situation, facilitating easier retrieval and manipulation of information. They are akin to schemas or blueprints that organize knowledge into manageable chunks. Attributes of the represented concept are represented by slots which have facets and default values.

4.3 | Object-Oriented Analogy: Classes and Instances

4.4 | Inheritance Hierarchies

4.5 | Scripts and Semantic Networks

4.6 | Default Reasoning and Conditionals

4.7 | Problems with Schema Systems

4.8 | Frames in Practice: JSON-LD

Semantic Network Analysis (SemNA)

A Tutorial on Preprocessing, Estimating, and Analyzing Semantic Networks (Christensen and Kenett 2023)

5.1 | Motivation

Why Semantic Network Analysis?

Network science has become a popular approach to study language, memory, personality, and psychopathology [10]. Semantic memory can be modelled as a network of nodes (concepts) and edges (associations).

Verbal fluency is one of the most widely used neuropsychological measures for capturing memory retrieval [1].

The Core Problem (according to Christensen and Kenett 2023) is that no standardised pipeline exists for pre-processing verbal fluency data [4]. Different labs make different decisions which (can) lead to reliability threats that compound in later analysis steps.

Also, novice researchers lack accessible resources to get started.

What Can Semantic Networks Reveal?

The structure of semantic memory – not just its content – is a meaningful, measurable, and clinically relevant cognitive variable.

Application Domain	Finding
Creative cognition	Lower average shortest path length (ASPL) linked to greater creative ability [7]
Cognition changes	Older adults show smaller clustering coefficient (CC), possibly explaining cognitive slowing [11]
Diagnostics of neurodivergence	(Asperger) autists show “hyper-modular” semantic networks; rigid, over-partitioned knowledge [8]
Personality (traits)	High openness to experience → more interconnected, flexible semantic networks [3]
Psychopathology / Diagnostics	Network structure of semantic memory linked to schizotypy and other clinical traits [4]

Table 5.1: What semantic networks can reveal.

5.2 | Key Definitions

Definition 5.1 semantic memory

Long-term memory of word meanings, conceptual categories, and general world knowledge..

Definition 5.2 network/graph

A mathematical structure of nodes (vertices) connected by edges (links)..

Definition 5.3 verbal fluency task

A task to measure verbal fluency. Participants generate as many category members within 60 seconds as possible.

We remember the definitions of:

knowledge graph: A large-scale semantic network encoding entities, types, and relations.

semantic network: Nodes = concepts/lexical items. Edges = similarity/co-occurrence/associations.

Note 5.1 Node Types Depend on Task

The type of node in a **semantic network** depends on what cognitive task is being analyzed. For example, the node type **category exemplars** is used in **verbal fluency tasks** and the node type **association (node type)** is used for free association tasks.

Network Science

Definition 5.4 network science

todo.

According to Christensen and Kenett 2023, Network science methodologies provide a powerful computational approach for modeling cognitive structures such as **semantic memory** and **mental lexicon**.

In the context of cognitive systems, **semantic memory** is for concept organization and the **mental lexicon** is for word storage and word retrieval.

It is possible to measure certain aspects of **semantic networks** such as local clustering, global integration and community structure. Additional information about the terms (nodes) of the network can be extracted through this measuring. Measuring methods are:

Definition 5.5 clustering coefficient (CC)

Measures local clustering and through that how closely connected nodes (and the terms they stand for) are.

Definition 5.6 average shortest path length (ASPL)

Measures global integration and through that how many steps are necessary to get from one node to another. This gives information about the connection efficiency of the network.

Definition 5.7 modularity (Q)

Measures the modularity and through that how much/how clearly the network is segmented into subgroups.

Verbal fluency

“One popular method to estimate semantic networks is via verbal fluency data. Verbal fluency asks participants to generate category members in 60 seconds. Categories can be semantic (e.g., animals) or phonological (e.g., words

that start with ‘s’).”

(Christensen and Kenett 2023)

Frage 5.1 **Why is verbal fluency used?**

The verbal fluency tasks are quick administer and the measurement is stable across cultures as general subjects such as animal species, food or vehicles can be used as categories. Further, the structure is well defined

Example 5.1 To **Zu Definition ?? (verbal fluency)**

Instruction: “Name as many animals as you can in 60 seconds.”

Response: cat, dog, mouse, elephant, giraffe, tiger, ...

Dataset Overview:

- 516 participants (rows)
- 38 columns per participant
- Animal naming task (60 seconds)
- Groups: Low vs. High openness

Column Structure

1. Group (1 = Low, 2 = High)
2. Openness score
3. Participant ID
4. Animal responses (Animal 1 – Animal 35)

A dataset could then look like this:

Group	Openness	ID	Animal 1	Animal 2
1	42	P001	cat	dog
1	38	P002	dog	mouse
2	67	P003	elephant	tiger
2	71	P004	giraffe	lion

Name	Description	Example
Perseveration	Repeat of the same response	cat, dog, cat
Intrusion	Response from the wrong category	apple (in animal task)
Moniker	Colloquial or informal name	water bear (tardigrade)
Compound	Missing space between words	guineapig
String	Multiple responses written as one word	catdogmouse
Typo	Spelling mistake	elefant

Table 5.2: Common Data Errors in Verbal Fluency. Perseverations and intrusions are counted and extracted for analysis [4].

From Raw Data to Semantic Networks

Semantic networks are constructed from patterns of co-occurrence across participants. If two concepts are frequently produced together by many participants, a stronger edge is formed between them in the network.

5.3 | Problem Statement

“There is no standardised pipeline for preprocessing verbal fluency data. Different labs make heterogeneous analytical decisions that compound in the network estimation and statistical analysis steps, threatening reliability and reproducibility.”

(Christensen and Kenett 2023)

As mentioned before in **Motivation**, there are no standardised pipelines for (pre)processing verbal fluency data. Additionally, there are no accessible tools for non-experts and the best estimation method (todo: for what?) is unknown.

Open research questions are:

- How to standardise preprocessing?
- Which methods for group comparisons?
- How does structure relate to individual differences?

5.4 | Proposed Solution: the SemNA pipeline

“A step-by-step depiction of the Semantic Network Analysis (SemNA) pipeline: pre- processing, estimating, and analyzing networks.”

(Christensen and Kenett 2023)

5.5 | Network Measures

5.6 | Results

5.7 | Limitations & Discussion

5.8 | Discussion & Conclusion

CHAPTER 6

The Resource Description Framework

6.1 | What is RDF?

Slides 1–4

Resource description frameworks is intended for situations where information on the Web needs to be processed by applications.

Definition 6.1 resource description framework

A framework for expressing information about resources. Resources can be anything, including documents, people, physical objects, and abstract concepts.

It provides a common framework for exchanging information without loss of meaning and enables **linked data** for publishing and interlinking data on the web.

The W3C (World Wide Web Consortium) standardizes the **resource description framework**.

6.2 | Triple Model

Slides 5–8

Definition 6.2 RDF triple model

Structure for an RDF statement where every piece of information is broken down into three parts: subject, predicate and object.

- **subject:** who or what we are talking about
- **predicate:** the relationship or property
- **object:** the thing the subject is related to

Example 6.1 todo

The same resource can appear in multiple triples, leading to a **NETWORK** of connected information which makes RDF powerful.

Example 6.2 TODO Mona Lisa

6.3 | International Resource Identifiers (IRIs/URIs)

Slides 8–9

Definition 6.3 international resource identifier
todo.

Example 6.3 To **Zu Definition 6.3 (international resource identifier)**

URLs are one form of IRI:

␣␣␣␣`http://dbpedia.org/resource/Leonardo_da_Vinci`

␣␣␣␣`http://xmlns.com/foaf/0.1/knows`

␣␣␣␣`http://www.example.org/bob#me`

International resource identifiers can appear in subject, predicate, or object position: **literals** are restricted to object position. International resource identifiers are also global identifiers, which means other people can reuse them. Resource description framework is gnostic about what an international resource identifier represents, only vocabularies provide meaning.

Example 6.4 Vocabularies

FOAF, Dublin Core, or Schema.org

6.4 | Literals

Slides 10–12

Definition 6.4 literal

Basic Data values that are not international resource identifiers.

Example 6.5 To **Zu Definition 6.4 (literal)**

strings, dates, numbers

Notation 6.1 The `datatype` symbol means “has datatype”.

Example 6.6 Literal with data type:

`"1990-07-04" <http://www.w3.org/2001/XMLSchema#date >`

Literal with language tag:

`"Leonard de Vinci"@fr`

Note 6.1 **literals** may only appear in the object position if a triple.

Example 6.7 Correct:

`:bob schema:birthDate "1990-07-04"sd:date .`

Is interpreted as: Bob’s birth date is 1990-07-04

Incorrect:

`"1990-07-04" schema:birthDate :bob`

Is interpreted as: 1990-07-04 has birthDate Bob, because **literals** are values (like text, numbers, dates), not entities that can act as subjects.

6.5 | Standard Format Turtle - Terse RDF Triple Language

Slides 13–16

6.6 | Standard Format JSON

Slides 17–19

6.7 | Flat Triplet Approach to RDF Graphs in JSON

Slides 20–24

6.8 | Comparison of Standard RDF Formats

Slides 25–27

6.9 | Summary

Slides 28–29

CHAPTER 7

Knowledge Graph Reasoning

7.1 | Motivation and Background

Problem 7.1 Incomplete Knowledge Graphs

Most existing **knowledge graphs** suffer from limitations in knowledge sources. Biases during knowledge extraction lead to incomplete or erroneous information.

Example 7.1 A **knowledge graph** about football might know “Nunez plays for Liverpool” but miss “Nunez is a forward”

Definition 7.1 Knowledge Graph Reasoning

Methodology that enables the generation of new facts from existing ones within a KG.

Application 7.1 ■ Intelligent question-answering

- e.g. “Who won the Champions League in 2020?”
- Recommendation systems
 - e.g. “Users who liked X also liked Y”
- Dialogue generation
 - e.g. Chatbots with world knowledge

	Embedding-Based Methods	Logical Rule-Based Methods
Interpretability	Lack it	Strong
TODO		

Key insight 7.1 Logical rule-based methods excel at inference by uncovering underlying logical rules, showcasing remarkable generalization ability and interpretability. They can be seamlessly integrated with neural networks.

Figure 7.1: Taxonomy of Logical Rule-Based **Knowledge Graph Reasoning** methods

Category		Key Characteristics	Timeline
Inductive Programming-Based	Logic	Inductive Logic Programming, Horn rules, rule learning from examples	1990s
Probabilistic Graphical Models + Rules		MLN (Markov Logic Networks), ProbLog, PSL (Probabilistic Soft Logic), fuzzy reasoning	2000s
Embedding Techniques + Rules	Tech-	KGE (Knowledge Graph Embedding), rule embedding, mutual enhancement	2010s
Neural Networks + Rules		RNN (Recurrent Neural Network), GNN (Graph Neural Network), neural theorem proving	2018+

7.2 | Background Knowledge

Definition 7.2 knowledge graph (formal)

A tuple $KG = (E, R, F)$ with E = a set of entities (nodes), R = set of relations (edges) and F = set of facts (triples).

Definition 7.3 fact (formal)

A triple $\langle e_h, r, e_t \rangle$ where $e_h \in E$ ist the head entity, $e_t \in E$ the tail entity and $r \in R$ a relation.

Example 7.2 Zu Definition 7.3 (fact (formal))

$\langle Nunez, PlayForTeam, Liverpool \rangle$

Figure 7.2: An example of a knowledge graph showing entities (circles) and relations (edges)

Definition 7.4 knowledge graph denoising

todo.

Figure 7.3: KGR task illustration

7.3 | Inductive Logic Programming-Based KGR Methods

Definition 7.5 Inductive Logic Programming

A rule-learning technique derived from first-order logic. It identifies logical programs, rules, or formulas shared across a dataset..

7.4 | Probabilistic Graphical Models + Rules

The core idea of unifying probabilistic graphical models and logical rules is to combine the expressive power of first-order logic with probabilistic reasoning to handle uncertainty and incompleteness in knowledge graphs.

Three Main Approaches:

Definition 7.6 Markov Logic Networks

todo Folie 19.

Definition 7.7 inference tree-based selective linear definite

todo.

Definition 7.8 Probabilistic Soft Logic

todo Folie 19.

Challenge 7.1 These methods rely on rule-based search mining requiring complex traversal computations and are therefore not well-suited for large-scale knowledge graphs.

7.5 | Embedding Techniques + Rules

7.6 | Neural Networks + Rules

Why should we combine NNs with rules?

NNs are fast, accurate, robust and good at generalization. But they are limited due to being black-boxes and lacking explainability. And our goal ist efficiency and explainable reasoning.

Approach	Dominant Role
Rule-Enhanced NN Modeling	NNs (rules augment NN)
NN-Enhanced Rule Learning	Rules (NNs help learn rules)

Table 7.1: The two subcategories of joining neural networks and logical rules.

Definition 7.9 Logic Attention Networ

todo.

Definition 7.10 Logic Attention Weight

A normalized logical relevance score that measures how strongly relation r implies query relation q , compared with the strongest implication coming from neighboring relations of entity e_i

Schreibweise: $\alpha_{j|i,q}^{Logic} = \frac{\mathcal{P}(r \Rightarrow q)}{\max(\{\mathcal{P}(r' \Rightarrow r) | r' \in N_R(e_i) \wedge r' \neq r\})}$

7.7 | Comparison and Analysis

Category	Method	Mean Reciprocal Rank	hit1	hit10
Inductive Logic Programming-Based	AnyBURL	0.310	0.233	–
Inductive Logic Programming-Based	SAFRAN	0.389	0.298	0.537
Embedding + Rules	RUGE	0.768	0.703	0.815
Embedding + Rules	TARE	0.781	0.721	0.839
NN + Rules	VN	0.463	0.345	0.526
NN + Rules	GCR	0.492	–	0.490
NN + Rules	GraIL	–	–	0.641
NN + Rules	CoMPiLE	–	–	0.847
NN + Rules	ConGLR	0.741	–	0.830
NN + Rules	BERTRL	0.718	–	0.867

Table 7.2: Performance Comparison on FB15K-237

Category	Accuracy	Efficiency	Interpretability	Transferability
Inductive Logic Programming-Based	Low–Medium	Low–Medium	High	Low
Probabilistic + Rules	Low	Low	Medium	Low
Embedding + Rules	Medium–High	High	Medium	Medium
NN + Rules	High	High	Medium–High	High

Table 7.3: Comparative Analysis Summary.

Key insight 7.2

- Inductive Logic Programming-based methods are most interpretable, but least efficient
- NN+Rules results in the best accuracy and efficiency, but interpretability still challenging
- The tradeoff is Accuracy/Efficiency vs. Interpretability

7.8 | Challenges and Future Directions

Challenge 7.2

Efficiency of Rule Learning Path search complexity grows exponentially with KG size and effective sampling strategies or pre-trained models are needed.

Challenge 7.3

Rule Quality Evaluation Traditional metrics (confidence, support) assume closed world (see Definition ?? (Open World Assumption)). Partial Completeness Assumption is a compromise, not definitive solution and Open World Assumption prevalent in KGs makes missing data unusable as negative examples.

Challenge 7.4

Interpretability Balance neural network integration improves speed but reduces interpretability. Embed reasoning steps in model architecture are needed.

Future Research Directions

1. Better Benchmark Datasets
 - Current datasets (FB15K-237, NELL-995, WN18R) do not capture all logical patterns.
 - Need datasets with pre-defined rules for proper evaluation.
2. Temporal KG (TKG) Reasoning
 - Uncover time-dependent or dynamic rules.
 - Combine RNN/LSTM with existing rule-learning methods.
3. Multi-Modal KG (MMKG) Reasoning
 - Heterogeneous information (images, text, etc.).
 - Use multi-modal learning for automatic rule extraction.
4. Few-Shot and Zero-Shot Rule Discovery
 - Enhance representation with additional information.
 - Leverage existing high-quality KGs as priors.
5. Incorporating Logical Rules into LLMs (Large Language Models)
 - Address LLM hallucinations.
 - Enhance pre-training, during-training, and post-training stages.

Key Takeaways

1. Knowledge graphs are often incomplete → reasoning is essential for filling gaps and correcting errors.
2. Four main categories of logical rule-based KGR:
 - Inductive Logic Programming
 - Probabilistic + Rules
 - Embedding + Rules
 - NN + Rules
3. Evolution:
 - From interpretable but inefficient (Inductive Logic Programming)
 - To efficient but less interpretable (NN + Rules)
4. Trade-off:
 - Accuracy/Efficiency vs. Interpretability remains a key challenge.
5. Best performance is currently achieved by NN + Rules hybrid methods.
6. Future directions:
 - Integration with LLMs
 - TKG/MMKG reasoning
 - Better evaluation benchmarks

CHAPTER 8

Knowledge Aquisition from Data I

CHAPTER 9

Knowledge Aquisition from Data II

Part II

Knowledge Graphs

CHAPTER 10

Start into semantic networks and knowledge graphs reasoning

CHAPTER 11

The Resource Description Framework (RDF)

CHAPTER 12

Knowledge Graph Reasoning

CHAPTER 13

Knowledge Acquisition from Data

CHAPTER 14

Representation Learning

Part III

Knowledge Representation in Machine Learning

CHAPTER 15

Knowledge Graph Embeddings

CHAPTER 16

Graph Neural Networks for Knowledge Representation

Part IV

Applications

CHAPTER 17

Knowledge Tracing

CHAPTER 18

Knowledge in Large Language Models

CHAPTER 19

KG- Aware Applications

Backmatter

Definitions

abduction todo. 29

abductive inference Reasoning to arrive at the best explanation. From observations to hypothetical causes. 15

association todo. 36

atom todo. 30

atomic formula todo. 25

average shortest path length Measures global integration and through that how many steps are necessary to get from one node to another. This gives information about the connection efficiency of the network. 36

bound todo. 26

captured todo. 26

category exemplars todo. 36

clause todo. 27

clustering coefficient Measures local clustering and through that how closely connected nodes (and the terms they stand for) are. 36

compound formula todo. 25

compound term todo. 30

Conjunctive Normal Form todo. 27

containment testing todo. 29

data todo. 10

declarative knowledge “Knowing **that**”. Factual and symbolic knowledge used for communication and abstract reasoning.. 33

declarative representation Declarative representations describe facts about the world as logical statements. 12

deduction todo. 29

derivability relation todo. 28

domain todo. 26

Existential Instantiation todo. 27

explicit knowledge Knowledge that is formally encoded in symbols, rules or graphs and can be processed by machines. 10

fact (formal) A triple $\langle e_h, r, e_t \rangle$ where $e_h \in E$ is the head entity, $e_t \in E$ the tail entity and $r \in R$ a relation. 44

fact (prolog) todo. 31

first-order logic todo. iv, 11, 25, 34, 44

formula todo. 25

frame Concept designed to provide a structured way to capture the essential aspects of a situation, facilitating easier retrieval and manipulation of information. They are akin to schemas or blueprints that organize knowledge into manageable chunks. Attributes of the represented concept are represented by **slots** which have facets and default values. 14, 34

frame-based representation todo. 12

free todo. 26

hit k proportion of accurate predictions among top-k outcomes. 45

horn clause todo. 30

identity of a functor todo. 31

inductive inference Generalising examples into patterns or rules. 15

Inductive Logic Programming A rule-learning technique derived from **first-order logic**. It identifies logical programs, rules, or formulas shared across a dataset.. 4, 44–47

inference todo. 27

inference tree-based selective linear definite todo. 45

information todo. 10

international resource identifier todo. v, 27, 40

knowledge todo. 10

Knowledge Acquisition Processes The process of incorporating knowledge into the knowledge base through strategies such as Interactive sessions with experts or Automatic generation from experience. 17

knowledge base todo. vi, 11

knowledge graph Graph-based knowledge representation where the entities are nodes and the relations are edges. Facts (subject, a predicate, and an object) are represented as triples.. 18

knowledge graph A large-scale semantic network encoding entities, types, and relations.. 17, 18, 22, 36, 43–45

knowledge graph (formal) A tuple $KG = (E, R, F)$ with E = a set of entities (nodes), R = set of relations (edges) and F = set of facts (triples). 44

knowledge graph denoising todo. 44

Knowledge Graph Reasoning Methodology that enables the generation of new facts from existing ones within a KG. 43

Knowledge Representation KR is a central problem in Artificial Intelligence that focuses on developing sufficiently precise notations for representing knowledge. 9, 19

- language structure** todo. 26
- linked data** todo. 39
- literal** Basic Data values that are not **international resource identifiers**. 27, 40, 41
- Logic Attention Networ** todo. 45
- Logic Attention Weight** A normalized logical relevance score that measures how strongly relation r implies query relation q , compared with the strongest implication coming from neighboring relations of entity e_i . 45
- logical inference** Inference that uses formal deduction from known facts. 14
- logical representation scheme** Representation schema based on formal logic and represents knowledge through logical formulas. It is strongly based on semantics and inference. 12
- Markov Logic Networks** todo Folie 19. 45
- Mean Reciprocal Rank** todo. 45
- mental lexicon** todo. 36
- modularity** Measures the modularity and through that how much/how clearly the network is segmented into sub-groups. 36
- Modus Ponens** todo. 27
- Modus Tollens** todo. 27
- multi-hop** todo. 20
- Natural Deduction** todo. 27
- Natural Language Processing** todo. 17
- network science** todo. 36
- network/graph** A mathematical structure of nodes (vertices) connected by edges (links).. 36
- neural network** todo. 46
- number (prolog)** todo. 30
- one-hop** todo. 20
- ontology** todo. 10
- Open World Assumption** Only things that are negated are false; everything else is unknown.. 46
- Partial Completeness Assumption** todo. 46
- preferred inference** Reasoning based on typical assumptions, subject to certain reservations. 15
- Prenex Normal Form** todo. 27
- Probabilistic Soft Logic** todo Folie 19. 45
- procedural knowledge** “Knowing **how**”. Compiled and action-oriented knowledge dependent on structural inter-connections. 33

procedural representation Representation schema that encodes knowledge as procedures or programmes that describe how to use that knowledge. It is efficient but hard to interpret. [12](#), [13](#)

Prolog todo. [29](#)

propositional Logic todo. [25](#)

quantified formula todo. [25](#)

query (prolog) todo. [31](#)

query folding todo. [30](#)

RDF triple model Structure for an [RDF](#) statement where every piece of information is broken down into three parts: subject, predicate and object. [39](#)

representation scheme A representation scheme is a formal notation used to define a [knowledge base](#) (Mylopoulos 1980). A knowledge base contains facts and rules and serves as a model of the real world. A representation scheme is therefore the ‘language’ in which we express knowledge, much like programming languages define the syntax and semantics in which we write instructions.. [11](#)

resolution todo. [27](#)

Resolution-based prover todo. [27](#)

resource description framework A framework for expressing information about resources. Resources can be anything, including documents, people, physical objects, and abstract concepts. [vi](#), [39](#), [40](#)

rule (prolog) todo. [31](#)

satisfiable todo. [26](#)

semantic memory Long-term memory of word meanings, conceptual categories, and general world knowledge.. [35](#), [36](#)

semantic network Nodes = concepts/lexical items. Edges = similarity/co-occurrence/associations.. [11](#), [35](#), [36](#)

semantic representation scheme Graph-based representations in which nodes represent entities and edges represent relationships between them. It supports inheritance and classification. [13](#)

Semantic Search todo. [18](#)

sentence todo. [26](#)

Sequent Calculus todo. [27](#)

slot todo. [iv](#), [14](#), [34](#)

specialized procedures Fixed inference patterns, which are often implemented as routines. [15](#)

structural knowledge “Knowing **why**”. Knowledge of how concepts within a domain are interrelated. This mediates declarative to procedural knowledge. [33](#)

substitution todo. [26](#)

test todo. [17](#)

Universal Instantiation todo. [27](#)

unsatisfiable todo. 26

valid todo. 26

variable (prolog) todo. 30

verbal fluency todo. vii, 35–37

verbal fluency task A task to measure **verbal fluency**. Participants generate as many category members within 60 seconds as possible. 36, 37

view differentiation todo. 29

Bibliography

- [1] Alfredo Ardila, Feggy Ostrosky-Solís, and Byron Bernal. “Cognitive testing toward the future: The example of semantic verbal fluency (ANIMALS)”. In: *International Journal of Psychology* 41.5 (2006), pp. 324–332. DOI: [10.1080/00207590500345542](https://doi.org/10.1080/00207590500345542).
- [2] Jiahang Cao et al. “Knowledge Graph Embedding: A Survey from the Perspective of Representation Spaces”. In: *ACM Comput. Surv.* 56.6 (Mar. 2024). ISSN: 0360-0300. DOI: [10.1145/3643806](https://doi.org/10.1145/3643806). URL: <https://doi.org/10.1145/3643806>.
- [3] Alexander P. Christensen, Yoed N. Kenett, et al. “Remotely close associations: Openness to experience and semantic memory structure”. In: *European Journal of Personality* 32.4 (2018), pp. 480–492. DOI: [10.1002/per.2157](https://doi.org/10.1002/per.2157).
- [4] Alexander P. Christensen and Yoed N. Kenett. “Semantic Network Analysis (SemNA): A Tutorial on Preprocessing, Estimating, and Analyzing Semantic Networks”. In: *Psychological Methods* 28.4 (2023), pp. 860–879. DOI: [10.1037/met0000463](https://doi.org/10.1037/met0000463).
- [5] Randall Davis, Howard Shrobe, and Peter Szolovits. “What Is a Knowledge Representation?” In: *AI Magazine* 14.1 (1993), pp. 17–33. DOI: <https://doi.org/10.1609/aimag.v14i1.1029>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1609/aimag.v14i1.1029>. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1609/aimag.v14i1.1029>.
- [6] Aidan Hogan et al. “Knowledge Graphs”. In: *ACM Comput. Surv.* 54.4 (July 2021). ISSN: 0360-0300. DOI: [10.1145/3447772](https://doi.org/10.1145/3447772). URL: <https://doi.org/10.1145/3447772>.
- [7] Yoed N. Kenett, David Anaki, and Miriam Faust. “Investigating the structure of semantic networks in low and high creative persons”. In: *Frontiers in Human Neuroscience* 8 (2014), p. 407. DOI: [10.3389/fnhum.2014.00407](https://doi.org/10.3389/fnhum.2014.00407).
- [8] Yoed N. Kenett, Rozanna Gold, and Miriam Faust. “The hyper-modular associative mind: A computational analysis of associative responses of persons with Asperger syndrome”. In: *Language and Speech* 59.3 (2016), pp. 297–317. DOI: [10.1177/0023830915589397](https://doi.org/10.1177/0023830915589397).
- [9] *Knowledge Representation 01 - Knowledge Basic Introduction, Motivation, History*. Vol. 1. Lecture Notes Knowledge Representation.
- [10] Cynthia S. Q. Siew et al. “Cognitive network science: A review of research on cognition through the lens of network representations, processes, and dynamics”. In: *Complexity* 2019 (2019), pp. 1–24. DOI: [10.1155/2019/2108423](https://doi.org/10.1155/2019/2108423).
- [11] Dirk U. Wulff, Thomas Hills, and Rui Mata. “Structural differences in the semantic networks of younger and older adults”. In: *PsyArXiv* (2018). DOI: [10.31234/osf.io/s73dp](https://doi.org/10.31234/osf.io/s73dp).